

C# Entry-Level Interview Questions and Detailed Answers (Week 1–7)

1. What is C# and where is it used?

C# is a modern, object-oriented, and type-safe programming language developed by Microsoft. It runs on the .NET platform and is used to build web applications (ASP.NET), desktop apps (WPF/WinForms), cloud services, APIs, microservices, and enterprise-grade systems.

2. What is CLR?

CLR (Common Language Runtime) is the execution engine of .NET. It manages memory allocation, garbage collection, exception handling, security, and JIT compilation of Intermediate Language (IL) into native code.

3. Difference between .NET Framework and .NET Core?

.NET Framework is Windows-only and tightly coupled to the OS. .NET Core (now unified as .NET) is cross-platform, modular, open-source, and optimized for cloud and microservices.

4. Value types vs Reference types?

Value types store actual data in stack memory (int, double, struct). Reference types store a reference pointing to heap memory (class, array, string). Value types are faster for small data; reference types support complex objects.

5. What are CTS and CLS?

CTS (Common Type System) defines how types are declared and managed in .NET. CLS (Common Language Specification) ensures interoperability between different .NET languages.

6. var vs dynamic vs explicit type?

var is compile-time type inference. dynamic is resolved at runtime and bypasses compile-time checking. Explicit typing declares the type manually and is fully compile-time checked.

7. What is boxing and unboxing?

Boxing converts a value type into an object (heap allocation). Unboxing extracts the value type from the object. Boxing introduces performance overhead.

8. Difference between == and Equals()?

== compares references by default for objects (unless overloaded). Equals() compares values logically and can be overridden for custom comparison.

9. string vs StringBuilder?

string is immutable; every modification creates a new object. StringBuilder is mutable and efficient for multiple string modifications.

10. What are nullable types?

Nullable types allow value types to store null using '?' (e.g., int?). Internally represented as Nullable.

11. Explain Access modifiers.

Access modifiers is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

12. Explain const vs readonly.

const vs readonly is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

13. Explain ref vs out.

ref vs out is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

14. Explain Garbage collection.

Garbage collection is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

15. Explain OOP pillars.

OOP pillars is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

16. Explain Encapsulation.

Encapsulation is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

17. Explain Abstraction.

Abstraction is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

18. Explain Inheritance.

Inheritance is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

19. Explain Polymorphism.

Polymorphism is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

20. Explain Method overloading.

Method overloading is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

21. Explain Method overriding.

Method overriding is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this

concept ensures better code quality, maintainability, and performance.

22. Explain virtual/override/new.

virtual/override/new is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

23. Explain Static method overriding.

Static method overriding is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

24. Explain Sealed class.

Sealed class is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

25. Explain Abstract class vs Interface.

Abstract class vs Interface is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

26. Explain Interface implementation.

Interface implementation is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

27. Explain Multiple interfaces.

Multiple interfaces is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

28. Explain Constructor overloading.

Constructor overloading is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

29. Explain Constructor chaining.

Constructor chaining is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

30. Explain Static constructor.

Static constructor is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

31. Explain Constructor inheritance.

Constructor inheritance is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

32. Explain this vs base.

this vs base is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

33. Explain Object initialization.

Object initialization is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

34. Explain Dependency inversion.

Dependency inversion is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

35. Explain Class vs Struct.

Class vs Struct is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

36. Explain Struct usage.

Struct usage is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

37. Explain Auto properties.

Auto properties is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

38. Explain Get and Set.

Get and Set is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

39. Explain Read-only properties.

Read-only properties is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

40. Explain Indexer.

Indexer is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

41. Explain Static class.

Static class is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

42. Explain Partial class.

Partial class is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

43. Explain Object initializer.

Object initializer is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

44. Explain Array vs List.

Array vs List is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

45. Explain ICollection.

ICollection is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

46. Explain IEnumerable.

IEnumerable is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

47. Explain IEnumerable vs ICollection.

IEnumerable vs ICollection is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

48. Explain Dictionary.

Dictionary is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

49. Explain Duplicate keys.

Duplicate keys is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

50. Explain List vs HashSet.

List vs HashSet is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

51. Explain Stack vs Queue.

Stack vs Queue is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

52. Explain Generics.

Generics is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

53. Explain Why generics.

Why generics is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

54. Explain Generic method.

Generic method is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

55. Explain Variance.

Variance is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

56. Explain ArrayList vs List.

ArrayList vs List is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

57. Explain Sorting.

Sorting is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

58. Explain Remove duplicates.

Remove duplicates is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

59. Explain Exception.

Exception is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

60. Explain Compile vs runtime error.

Compile vs runtime error is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

61. Explain try-catch.

try-catch is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

62. Explain finally.

finally is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

63. Explain finally skip.

finally skip is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

64. Explain throw vs throw ex.

throw vs throw ex is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

65. Explain Custom exception.

Custom exception is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

66. Explain Base exception.

Base exception is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

67. Explain Stack trace.

Stack trace is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

68. Explain Create exception.

Create exception is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

69. Explain Delegate.

Delegate is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

70. Explain Delegate vs method.

Delegate vs method is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

71. Explain Multicast delegate.

Multicast delegate is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

72. Explain Func/Action.

Func/Action is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

73. Explain Lambda.

Lambda is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

74. Explain Delegate vs Event.

Delegate vs Event is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

75. Explain Event subscription.

Event subscription is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

76. Explain Anonymous method.

Anonymous method is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

77. Explain Action vs Func.

Action vs Func is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

78. Explain Predicate.

Predicate is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

79. Explain LINQ.

LINQ is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

80. Explain LINQ syntax.

LINQ syntax is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

81. Explain Deferred execution.

Deferred execution is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

82. Explain Immediate execution.

Immediate execution is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

83. Explain Where vs Select.

Where vs Select is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

84. Explain First vs FirstOrDefault.

First vs FirstOrDefault is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

85. Explain GroupBy.

GroupBy is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

86. Explain OrderBy vs ThenBy.

OrderBy vs ThenBy is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

87. Explain Any vs All.

Any vs All is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

88. Explain Projection.

Projection is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

89. Explain FileStream.

FileStream is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

90. Explain StreamReader/Writer.

StreamReader/Writer is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

91. Explain BOM.

BOM is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

92. Explain Read vs ReadLine.

Read vs ReadLine is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

93. Explain Dispose streams.

Dispose streams is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

94. Explain Thread.

Thread is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

95. Explain Process vs Thread.

Process vs Thread is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

96. Explain Race condition.

Race condition is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

97. Explain lock keyword.

lock keyword is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.

98. *Explain Deadlock.*

Deadlock is an important concept in C#. A strong explanation should include its definition, purpose, syntax, practical use cases, and implications in real-world applications. Understanding this concept ensures better code quality, maintainability, and performance.